

A PROFESSIONAL TEXTBOOK

System Design From First Principles to Staff Engineer

A build-from-scratch textbook that teaches distributed systems in plain English — every concept explained with the what, why, when, how, and the trade-offs that decide real designs.

VOLUME I — Foundations & the Building Blocks

First principles

Real architectures

Interview-ready

Diagrams throughout

A living textbook, built and delivered chapter by chapter. This volume opens with the complete book blueprint and Chapter 1 in full.

PREFACE

How this book is built

An honest word on how to get the most from it.

This book assumes you can already program but have never formally studied system design. It starts every topic from zero, defines every term, and never asks you to accept a design on faith — you will always see the problem first, then why simpler answers fail, then how the real solution works, and finally when *not* to use it.

The teaching pattern in every chapter

To build real judgment (not memorized answers), every concept is taught in the same four beats:

Beat	The question it answers
1. What problem?	What real pain does this concept exist to solve?
2. Why did older ways fail?	What did people try first, and where did it break?
3. How does it work?	The mechanism, step by step, with diagrams.
4. When NOT to use it?	Every tool has a cost. Knowing its limits is the senior skill.

Each chapter also carries the full study apparatus you would expect from a university text: learning objectives, a real-world analogy, architecture and flow diagrams, common mistakes, interview questions, hands-on exercises, a mini project, a large project, and a summary.

HOW TO ACTUALLY LEARN FROM THIS

Read actively. When a chapter poses a design problem, cover the solution and attempt it yourself on paper for five minutes first. The **gap** between your attempt and the book's answer — and the reasons for the difference — is where engineering judgment forms. Passive reading builds recognition; active designing builds skill.

A NOTE ON SCOPE & DELIVERY

This is a large work delivered **chapter by chapter**. The blueprint that follows is the complete plan; each chapter is then written in full as its own printable unit. Study a chapter, then request the next. Built this way, the material stays deep and accurate instead of thinning into filler.

BLUEPRINT

The complete book

Ten parts, seventy chapters — the whole journey, mapped.

Your entire topic list, organized into a coherent learning path. Each line becomes a full chapter with the apparatus described in the Preface. Chapter 1 follows immediately.

PART I — Foundations

- 1 Introduction to System Design ← **in this volume**
- 2 How Computers Work: CPU, Memory, Storage
- 3 Networking Fundamentals: TCP, UDP, IP
- 4 The Web: HTTP, HTTPS, DNS
- 5 Latency, Throughput & the Numbers Every Engineer Knows

PART II — Building Blocks of Scalable Systems

- 6 Client-Server Model & the Request Lifecycle
- 7 Load Balancing & Reverse / Forward Proxies
- 8 Caching: CDN, Redis, Memcached
- 9 API Gateways & the Edge
- 10 Vertical vs Horizontal Scaling

PART III — Data & Storage

- 11 Relational Databases & SQL
- 12 Indexing & Query Performance
- 13 Normalization, Denormalization & Data Modeling
- 14 Transactions, ACID & Isolation Levels
- 15 NoSQL: Key-Value, Document, Wide-Column, Graph
- 16 Replication
- 17 Partitioning & Sharding
- 18 Consistent Hashing

PART IV — Distributed Systems Theory

- 19 Fallacies & Hard Truths of Distributed Systems

- 20 CAP Theorem & PACELC
- 21 Consistency Models: Strong, Eventual, Causal
- 22 Consensus (Paxos, Raft) & Coordination (ZooKeeper)
- 23 Time, Clocks, Ordering & Distributed Locking
- 24 Unique IDs at Scale: UUID & Snowflake

PART V — Communication & Architecture Styles

- 25 REST & API Design
- 26 gRPC & Protocol Buffers
- 27 GraphQL
- 28 WebSockets & Real-Time Communication
- 29 Monolith vs Microservices
- 30 Service Mesh
- 31 Message Queues & Brokers: Kafka, RabbitMQ, SQS
- 32 Event-Driven Architecture & Pub/Sub
- 33 CQRS & Event Sourcing

PART VI — Reliability & Resilience Patterns

- 34 Availability, Reliability & the Nines
- 35 Rate Limiting & Throttling
- 36 Circuit Breaker, Retry, Timeout, Bulkhead
- 37 Idempotency & the Outbox Pattern
- 38 Saga Pattern & Distributed Transactions
- 39 Bloom Filters & Probabilistic Data Structures
- 40 Disaster Recovery, Backup & High Availability

PART VII — Security

- 41 Authentication: Sessions, Cookies, JWT
- 42 Authorization & OAuth 2.0 / OIDC
- 43 Encryption, TLS & Secrets Management
- 44 Securing Distributed Systems: Zero-Trust & mTLS

PART VIII — Infrastructure & Operations

- 45 Containers & Docker
- 46 Orchestration with Kubernetes
- 47 Cloud Computing: AWS, Azure, GCP
- 48 Infrastructure as Code: Terraform
- 49 CI/CD Pipelines
- 50 Observability: Monitoring, Logging, Tracing
- 51 Capacity Planning & Cost Optimization

PART IX — The Design Method & Interview

- 52 A Repeatable Framework for Any Design
- 53 Back-of-the-Envelope Estimation
- 54 Trade-off Analysis & Communicating a Design

PART X — Case Studies (Applied System Design)

- 55 URL Shortener (TinyURL)
- 56 Rate Limiter
- 57 Distributed Unique ID Generator
- 58 News Feed: Twitter/X, Instagram, Facebook, LinkedIn
- 59 Chat System: WhatsApp, Messenger
- 60 Notification System
- 61 Web Crawler & Google Search
- 62 Distributed Key-Value Store (Dynamo)
- 63 Video Streaming: YouTube, Netflix
- 64 Ride-Sharing (Uber) & Food Delivery
- 65 E-commerce & Inventory (Amazon)
- 66 Payment System, Banking & ATM
- 67 File Storage & Sync: Dropbox, Google Drive
- 68 Search & Recommendation Systems
- 69 Stock Exchange & Real-Time Dashboards
- 70 Modern AI Infrastructure: LLM Serving, Vector DBs, RAG

CHAPTER 1

Introduction to System Design

What it is, why it exists, and the method you will use for the rest of the book.

LEARNING OBJECTIVES

- Explain what system design is and how it differs from writing code.
- Tell functional and non-functional requirements apart, with examples.
- Define the core qualities of a system: scalability, reliability, availability, performance, maintainability, cost, and security — in plain English.
- Clearly distinguish latency from throughput, and vertical from horizontal scaling.
- Read an availability “nines” table and know what it means for real downtime.
- Apply a repeatable six-step framework to any design problem.
- Do a first back-of-the-envelope estimate using the numbers every engineer should know.

REAL-WORLD ANALOGY — DESIGNING A HIGHWAY SYSTEM

Writing a program is like building one good car. **System design is like designing the entire highway system** that thousands of cars will drive on at once. A single well-built car (good code) is not enough: if the roads have one lane, no exits, and no plan for rush hour, everything jams. The highway designer thinks about traffic volume at peak hour (load), how fast a trip takes (latency), how many cars pass per minute (throughput), what happens when one road floods (failure), and how to add lanes as the city grows (scaling). Throughout this book you are learning to be that highway designer — for software.

1.1 What is system design?

System design is the process of deciding how the parts of a software system fit together so that it meets its goals under real-world conditions — many users, unreliable networks, hardware that fails, and a limited budget. It is the high-level plan: which components exist (servers, databases, caches, queues), how they talk to each other, where the data lives, and what happens when something breaks.

It sits *above* the code. Writing a function that sorts a list is programming. Deciding that you need three copies of your database in different data centers, a cache in front of it, and a queue to absorb traffic spikes — that is system design. Both matter, but they are different skills, and this book is about the second one.

WHAT PROBLEM DOES SYSTEM DESIGN SOLVE?

A program that works perfectly on your laptop for one user can fall apart when a million people use it at the same time. It may become slow, lose data, or crash entirely. System design solves the problem of building software that **stays fast, correct, and available even under heavy load and partial failure** — and does so at a cost the business can afford.

1.2 Why simple approaches stop working

Almost every system starts the same simple way: one server running the application and its database together. A user's browser sends a request; the server does the work, reads or writes the database, and sends a response. For a small app, this is perfect — simple, cheap, easy to understand.

THE SIMPLE START (works great... until it doesn't)

```
[ user's browser ] --request--> [ ONE SERVER ]
                        <--response--  app + database
                                      on one machine
```

Then success arrives, and this simple design fails in predictable ways:

- **Too much traffic:** one machine can only handle so many requests per second. Past that, everyone waits, then requests start failing.
- **It becomes a single point of failure:** if that one machine dies, the *entire* product is down. There is no backup.
- **Data outgrows the disk:** one machine has limited storage and memory. Eventually the data no longer fits.
- **Everything is coupled:** a heavy report query can starve the database and slow down every user's simple actions.

The rest of this book is, in a sense, the story of fixing these four problems — adding more machines (scaling), copies for safety (replication), layers to absorb load (caching, queues), and ways to keep working when parts fail (resilience). Each solution brings its own new problems, which is exactly why system design requires judgment rather than a fixed recipe.

1.3 Requirements: functional vs non-functional

Before designing anything, you must know what it needs to do. Requirements come in two kinds, and confusing them is a classic beginner mistake.

Functional requirements	Non-functional requirements
What the system does — its features and behavior.	How well it does it — its qualities under load and failure.
"A user can post a photo."	"The feed loads in under 200 ms."
"A user can follow another user."	"The service is available 99.99% of the time."
"A user can search for products."	"It supports 10 million users."

KEY INSIGHT — THE NON-FUNCTIONAL REQUIREMENTS SHAPE THE ARCHITECTURE

Two apps can have the *same features* (functional requirements) yet need completely different designs because their **non-functional** requirements differ. A photo app for 100 friends and a photo app for 100 million users both "let a user post a photo," but only the second one forces caching, sharding, CDNs, and replication. When you design, the numbers — scale, latency, availability — are what pick your tools.

1.4 The core qualities of a system

Non-functional requirements are really about a handful of qualities. These words appear in every chapter, so learn them now in plain English.

Quality	Plain-English meaning	Why it matters
Scalability	The ability to handle more work by adding resources.	Growth shouldn't require a rewrite.
Reliability	The system does the correct thing, and doesn't lose or corrupt data.	Users trust it with their data and money.
Availability	The system is up and responding when users need it.	Downtime means lost users and revenue.
Performance	It responds quickly (latency) and handles many requests (throughput).	Slow products lose users.
Maintainability	Engineers can understand, change, and operate it safely.	Most of a system's life is spent being changed.
Cost	The money spent on servers, storage, bandwidth, and people.	An unaffordable design is not a design.
Security	Protecting data and access from misuse and attack.	One breach can end a company.

THERE IS NO PERFECT SYSTEM — ONLY CHOSEN TRADE-OFFS

These qualities pull against each other. More availability often costs more money. More consistency can cost performance. A cache adds speed but risks stale data. **You cannot maximize everything at once.** The whole craft of system design is choosing which qualities matter most for *this* system and deliberately trading away the rest. Any engineer who claims a design has no downsides simply hasn't found them yet.

1.5 Latency vs throughput

People mix these up constantly. They are different, and a good system needs both.

- **Latency** = how long *one* request takes, from start to finish. Measured in time (milliseconds). Lower is better.
- **Throughput** = how *many* requests the system handles per unit of time. Measured in operations per second (or requests/sec, QPS). Higher is better.

ANALOGY

Think of a supermarket checkout. **Latency** is how long *you personally* wait in line to be served. **Throughput** is how many customers the whole store checks out per hour. Adding more checkout lanes raises throughput (more customers served per hour) even if each customer's own wait (latency) stays the same. The two are related but not the same — and improving one does not automatically improve the other.

COMMON CONFUSION — AVERAGES LIE

Never judge latency by its average alone. What hurts users is the **tail**: the slowest 1% of requests, written **p99** (the 99th percentile — 99% of requests are faster than this number). A system with a great average but a terrible p99 still feels broken to many users. Always ask about p95 and p99, not just the mean.

1.6 Availability and the “nines”

Availability is the percentage of time a system is up and working. It is quoted in “nines,” and each extra nine is dramatically harder and more expensive to achieve. This table is worth memorizing:

Availability	Name	Downtime per year	Downtime per day
99%	“two nines”	~3.65 days	~14.4 minutes
99.9%	“three nines”	~8.77 hours	~1.44 minutes
99.99%	“four nines”	~52.6 minutes	~8.6 seconds
99.999%	“five nines”	~5.26 minutes	~0.86 seconds

RELIABILITY VS AVAILABILITY — NOT THE SAME THING

A system can be **available** (responding) but not **reliable** (giving wrong answers or losing data). Availability asks “is it up?” Reliability asks “is it correct?” You want both, but they are measured and engineered differently. A bank that is always online but occasionally loses a deposit is available yet catastrophically unreliable.

1.7 Two ways to scale: up vs out

When one machine is no longer enough, you have two choices.

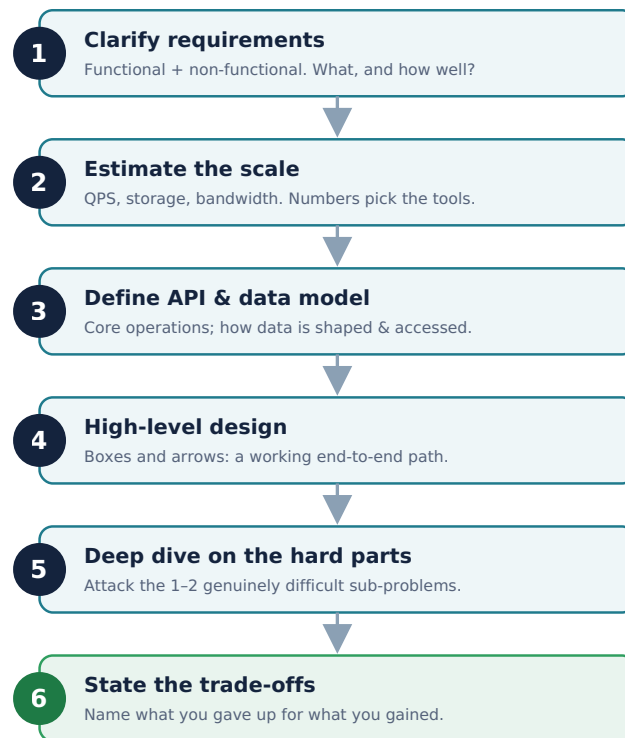
Vertical scaling (scale UP)	Horizontal scaling (scale OUT)
Make the single machine more powerful — more CPU, RAM, faster disk.	Add more machines and share the work among them.
+ Simple; no code changes. - A hard ceiling — you can only buy so big. - Still a single point of failure. - Cost rises steeply at the top end.	+ Near-unlimited growth — just add machines. + No single point of failure (with several machines). - Harder: needs load balancing, data distribution, and handling of machine failures.

WHY LARGE SYSTEMS SCALE OUT

Vertical scaling is the easy first step and is often the right choice early. But every serious large-scale system eventually scales **horizontally**, because it is the only path that removes the single-machine ceiling *and* the single point of failure at once. Most of this book’s hard problems — load balancing, replication, sharding, consistency — exist precisely because scaling out means coordinating many machines that can each fail independently.

1.8 A repeatable framework for any design

Faced with “design Twitter,” beginners panic and start naming technologies at random. Experts follow a calm, repeatable method. Use these six steps for every problem in this book and in interviews.



Notice the order: you understand the problem and its *scale* before drawing a single box, and you finish by naming trade-offs rather than pretending there are none. This method turns a scary open-ended prompt into six calm, ordered questions.

1.9 Back-of-the-envelope estimation

Step 2 above — estimating scale — relies on rough mental math with round numbers. You are not looking for precision; you are looking for the right *order of magnitude* so you know whether one machine will do or you need a thousand. Two things make this easy: powers of ten, and a few latency numbers.

Data sizes and the powers of ten

Unit	Size	Rough feel
KB (kilobyte)	1,000 bytes	A short text message.
MB (megabyte)	1,000 KB	A small photo or a minute of music.
GB (gigabyte)	1,000 MB	A movie; fits in RAM on one server.
TB (terabyte)	1,000 GB	Too big for one machine’s memory — think disks.
PB (petabyte)	1,000 TB	Fleet-of-machines scale (large companies’ data).

Latency numbers every engineer should know (approximate)

These are rounded orders of magnitude, not exact figures, but they tell you instantly why we cache, why we keep data close, and why crossing the planet is expensive.

Operation	Approx. time	Lesson
Read from main memory (RAM)	~100 nanoseconds	Memory is fast — keep hot data here.
Read 1 MB sequentially from RAM	~micros-tens of μ s	Bulk memory reads are cheap.
Solid-state disk (SSD) random read	~100 microseconds	SSD is ~1000 $\times$ slower than RAM.
Network round trip within a data center	~0.5 milliseconds	Same building is fine.
Spinning hard-disk seek	~10 milliseconds	Disks are slow — avoid random disk reads.
Network round trip across the world	~150 milliseconds	Distance costs time — use CDNs and replicas near users.

A WORKED MINI-ESTIMATE

Suppose an app gets **1 million writes per day**. Is that a lot? Divide by ~86,400 seconds in a day: about **~12 writes per second on average** — tiny, one machine handles it easily. But traffic isn't flat; peaks are often **~2-3 $\times$ the average**, so plan for ~30/sec. Now suppose each write stores ~1 KB. Over five years: $1\text{M} \times 365 \times 5 \times 1\text{ KB} \approx$ **~1.8 TB** — too big for memory, comfortably fine for disk, and a hint you may later shard. In three lines of arithmetic you already know the rough shape of the system. That is the entire point of estimation.

1.10 The map of this book

Everything ahead is an answer to one of the four failures from Section 1.2. Here is the shape of the journey, so each later chapter has a home in your mind.

```

the four failures of the "one server" ..... the book's answer
-----
too much traffic for one machine ..... load balancing, scaling out,
                                         caching, queues (Parts II, V, VI)
one machine dies = total outage ..... replication, high availability,
                                         disaster recovery (Parts III, VI)
data outgrows one disk ..... partitioning & sharding, NoSQL,
                                         consistent hashing (Part III)
everything is coupled & fragile ..... microservices, resilience patterns,
                                         observability (Parts V, VI, VIII)
tying it together ..... theory (Part IV), method (Part IX),
                                         real case studies (Part X)

```

Common mistakes (beginners make these)

WATCH OUT

- **Jumping to technology first** (“let’s use Kafka!”) before understanding requirements and scale.
- **Ignoring non-functional requirements** — designing features without asking about load, latency, or availability.
- **Optimizing prematurely** — adding sharding and caching to a system that has ten users. Complexity you don’t need is pure cost.
- **Confusing latency with throughput**, or judging latency by its average instead of p99.
- **Pretending a design has no downsides**. Every choice is a trade-off; not naming them is a red flag.
- **Designing for infinite scale on day one**. Start simple; scale when the numbers demand it.

Interview questions

1. What is the difference between functional and non-functional requirements? Give two examples of each for a ride-hailing app.
2. Explain latency and throughput. Can a system have low latency but low throughput? Give an example.
3. What does “four nines” of availability mean in minutes of downtime per year?
4. Compare vertical and horizontal scaling. Why do large systems eventually scale horizontally?
5. Why is average latency a misleading metric? What would you ask for instead?
6. Walk me through the steps you take when given an open-ended design problem.
7. A service receives 5 million requests per day. Estimate the average requests per second, and the likely peak.

Hands-on exercises

1. Write down three functional and three non-functional requirements for a simple note-taking app.
2. Convert these availability targets to yearly downtime by hand: 99%, 99.9%, 99.99%.
3. An app stores 2 KB per user event and expects 4 million events per day. Estimate storage growth per year.
4. For a photo-sharing app, list which of the seven core qualities you would prioritize, and which you would trade away, and why.
5. Draw the “one server” diagram, then redraw it with one improvement that removes the single point of failure. (You’ll formalize this in Part II.)

Mini project

DESIGN A PASTEBIN, ON PAPER

Using only the six-step framework from Section 1.8, sketch a design for a “Pastebin” (a service where a user pastes text and gets a short link to share it). Don’t worry about being right — just practice the *method*: write two functional and two non-functional requirements, estimate writes/reads per day if it has 1 million users, list the three or four boxes you’d need, and name one trade-off. Keep this sheet; you will compare it against the full URL-shortener case study in Chapter 55.

Large project (runs across the book)

“BUILD A SOCIAL PLATFORM” — YOUR RECURRING CAPSTONE

Start a design notebook for a social media platform (think a simplified Instagram). In each part of this book, you will add one layer: now (Part I) just write its requirements and a rough scale estimate (how many users, posts per day, reads per post). As you learn load balancing, caching, databases, sharding, feeds, and resilience, you will return and evolve this same design — ending the book with a complete, defensible architecture you designed yourself. This is the single most valuable thing you can do to turn reading into skill.

Chapter summary

- **System design** is the high-level plan for how a software system’s parts fit together to stay fast, correct, and available under real load and failure — at acceptable cost.
- The naive “**one server**” design fails in four ways: too much traffic, single point of failure, data outgrowing one disk, and tight coupling. The whole book answers these.
- **Functional** requirements say what a system does; **non-functional** requirements (scale, latency, availability) say how well — and they shape the architecture.
- The core qualities — scalability, reliability, availability, performance, maintainability, cost, security — **trade off against each other**; design is choosing among them on purpose.
- **Latency** is per-request time; **throughput** is requests per second. Judge latency by **p99**, not the average.
- Scale **up** (bigger machine) is simple but capped; scale **out** (more machines) is unlimited but hard — and is why distributed-systems problems exist.
- Use the **six-step framework** and **back-of-the-envelope estimation** on every problem, starting with the mini and large projects above.

COMING NEXT

Chapter 2 — How Computers Work: CPU, Memory, Storage. Before we connect many machines, we look inside one: why memory is fast and disks are slow, what actually limits a single server, and how those hardware truths drive every scaling decision that follows.

CONTINUING

How to get the next chapter

You now have the full blueprint and Chapter 1 in the exact format every chapter will follow. To keep building the book, simply reply with:

“Continue — write Chapter 2.”

Each continuation delivers the next full chapter (occasionally two shorter ones together) as its own printable PDF, in this same style, with all the study apparatus. You can also ask for adjustments at any time — more code examples in a specific language, deeper coverage of a topic, the editable Mermaid source for a chapter’s diagrams, or a faster pace with lighter chapters.

Because this is a genuinely book-length work, expect the complete text to span many sessions. Delivered this way, each chapter stays deep, accurate, and worth studying — which is the whole point.